

GIT COMMANDS MASTERCLASS

A Hands-On Learning Guide from Your Web Projects

Attribution, Workflows, Remotes, and Rollbacks

Based on real scenarios in connectagroharbour / agroharbour and webtantu

Developer Profile:

Ayush Kushwah (ayush-kushwah)
ayushkushwah1404@gmail.com

Generated on: May 2026

Designed to accelerate Git comprehension & professional conventions

Table of Contents

Section 1: Git Core Identity & Contributions	Page 3
Section 2: The Everyday Agile Workflow	Page 4
Section 3: GitHub Remotes & Authentication Mix-Ups	Page 5
Section 4: Safety Nets, History Inspecting, & Rollbacks	Page 6
Section 5: Real-World Scenarios from AgroHarbour & WebTantu	Page 7
Section 6: Interactive Command Reference Sheet	Page 8

Section 1: Git Core Identity & Contribution Tracking

Every single commit in Git is tagged with the developer's metadata: their Name and Email address. GitHub uses this email metadata to automatically associate your local commits with your GitHub profile, which in turn updates your contribution graph (lighting up the green activity grid squares).

If you do not configure your identity correctly, or if you use an email address that is not verified on your GitHub account, Git will still commit your code, but GitHub will display them under a blank profile or fail to count them on your graph.

1.1 Inspecting Your Current Configuration

To see what name and email address Git is currently using to stamp your commits:

```
git config user.name
git config user.email
```

If you want to check your global default configuration, append the `--global` flag:

```
git config --global user.name
git config --global user.email
```

1.2 Updating Your Identity

To set your local configuration to match your official GitHub username and verified email:

```
git config user.name "ayush-kushwah"
git config user.email "ayushkushwah1404@gmail.com"
```

PRO-TIP:

Setting local configs (without `--global`) is perfect when you work on work-related projects and personal projects on the same machine and need to use separate email addresses for each directory.

1.3 Amending the Most Recent Commit

Suppose you just made a commit, only to realize Git attributed it to the wrong name/email (e.g. webtantu instead of your own). Instead of deleting the commit or starting over, you can tell Git to overwrite the most recent commit's author using your newly updated config:

```
git commit --amend --reset-author --no-edit
```

Here, `--reset-author` forces Git to read your updated `user.name` and `user.email` configurations and apply them to the commit. The `--no-edit` flag tells Git to reuse the existing commit message so you don't have to retype it.

Section 2: The Everyday Agile Workflow

The fundamental lifecycle of git is: edit files in your working directory -> stage files to indicate readiness -> save them permanently to local history -> push them to GitHub. Mastering this flow is essential for incremental, safe, and collaborative coding.

2.1 Checking Status

Always make it a habit to check what files have changed before staging or committing. This prevents accidental inclusion of system files, build outputs, or sensitive database keys (like your local .env values!).

```
git status
```

2.2 Staging Targeted Changes

Staging is like placing items in a shipping box before taping it shut. You can stage everything in the directory:

```
git add .
```

Or better yet, stage specific files to keep your commits modular and tidy:

```
git add src/app/components/ContactSection.jsx public/robots.txt
```

2.3 Committing with Intent

A commit takes the staged changes and logs a permanent snapshot in your git history. Commit messages should describe **what** changed and **why** so other developers can understand the history at a glance.

```
git commit -m "fix: resolve mobile menu navigation scroll-to-top behavior"
```

2.4 Pushing to GitHub

This uploads your local commits to the remote repository ('origin') on the specified branch ('main'). Once pushed, services like Netlify or Vercel trigger automatically to build and deploy your live site!

```
git push origin main
```

Agile Habit:

Running git status frequently during your workflow acts like visual feedback. If a file is in red, it's modified but unstaged. If it's green, it's ready to be committed.

Section 3: GitHub Remotes & Authentication Mix-Ups

A common friction point for developers working on multiple GitHub profiles (e.g. standard accounts, agency, or different project usernames) is credential mismatch. This manifests as a 403 Forbidden error during git push.

3.1 Inspecting Remotes

To inspect the remote servers connected to your local directory:

```
git remote -v
```

This outputs something like:

```
origin https://github.com/connectagroharbour/agroharbour.git (fetch)
origin https://github.com/connectagroharbour/agroharbour.git (push)
```

3.2 Resolving '403 Forbidden' (Authentication Mismatch)

When you run git push, Windows Credential Manager automatically provides credentials for the host (github.com). If you have credentials saved for a previous profile (e.g., 'webtantu'), your computer tries to authenticate the push using that saved username. But if that repository ('agroharbour') only allows access to your main account ('ayush-kushwah'), GitHub rejects it with a '403 Forbidden / Permission denied' error.

The Solution: URL-based User Injection

Instead of deleting all stored credentials, we can explicitly specify which username Git must use for this project by updating the remote origin URL:

```
git remote set-url origin https://ayush-kushwah@github.com/connectagroharbour/agroharbour.git
```

By injecting 'ayush-kushwah@' directly into the URL, we bypass default fallback behaviors. When you run the next git push command, Git recognizes the specific username request, bypasses the incorrect cached credentials, and prompts a Windows/Browser login screen allowing you to sign in with the correct profile!

Alternative Fix:

On Windows, you can also open 'Credential Manager' from the Start Menu, click 'Windows Credentials', find 'git:https://github.com', and remove it. This forces Git to ask for login details on your next push.

Section 4: Safety Nets, History Inspecting, & Rollbacks

In production environments, having a system of safety checkpoints prevents regressions. If you build a highly complex feature in Phase 6 that breaks a form you perfected in Phase 3, you need reliable ways to check diffs or revert files safely.

4.1 Reviewing the Commit History

To inspect the list of previous commits, who made them, and when:

```
git log
```

To see a compact, single-line overview of the history:

```
git log --oneline -n 10
```

To inspect the exact details (author name and email) of the very last commit:

```
git log -1 --format="%an <%ae>"
```

4.2 Phase-Based Git Tagging

Tags are permanent bookmark tags for specific commits. They act as anchors for finished milestones, insulating you against later breaking changes. Setting tags at completion stages is a core industry practice.

```
git tag phase-3-complete
```

To list all existing tags in your project:

```
git tag
```

4.3 Safe Reverting and Diffs

To see the differences between your current code and a previous tag:

```
git diff phase-3-complete
```

If you completely break a file and want to revert it back to exactly how it was in Phase 3 (without losing other work):

```
git checkout phase-3-complete -- src/app/components/ContactSection.jsx
```

CRITICAL WARNING:

Never run 'git reset --hard' unless you are 100% sure you want to delete all local uncommitted changes! Using 'git checkout <tag> -- <file>' is infinitely safer because it targets only a single, specified file.

Section 5: Real-World Scenarios from AgroHarbour & WebTantu

Let's break down exactly what happened behind the scenes in your actual repositories, why we ran each command, and what the key learning takeaways are.

Case Study A: AgroHarbour Deployment & Refresh Errors

During our deployment of AgroHarbour, we ran into routing and redirection issues where refreshing the live SPA returned a 404 page. We created `vercel.json` and `_redirects` to solve this. We staged and deployed these safely using:

```
git status
git add .
git commit -m "fix: resolve 404 refresh error and finalize consultation form deployment"
git push origin main
```

Takeaway: The status check confirmed we weren't uploading private environment folders. The push triggered the Vercel CDN pipeline to deploy the code globally within minutes.

Case Study B: The GitHub Contribution Mismatch

In WebTantu, we noticed your contributions were not showing on your GitHub profile green-square calendar. We ran checks and verified your identity parameters:

```
git log -1 --format="%an <%ae>"
git config user.name; git config user.email
```

We realized the commits were made with your email address, but your profile configuration locally defaulted to another alias. We corrected this globally and amended the author of the commit before pushing to align the GitHub contribution systems.

Case Study C: The Permission Forbidden Mismatch (403)

We faced a push failure because the local system was authenticating commits with the 'webtantu' cached username. Since 'webtantu' had no push permissions to 'agroharbour', GitHub locked us out. We resolved it securely without wiping the credential manager database by telling origin to explicitly request your main account:

```
git remote set-url origin https://ayush-kushwah@github.com/connectagroharbour/agroharbour.git
git push origin main
```

Takeaway: Explicitly declaring 'username@' forces the Git network helper to isolate the authentication scope, letting you handle multiple developer identities safely.

Section 6: Interactive Command Reference Sheet

Print or save this cheat sheet to quickly lookup commands during development.

git init	Initializes a blank Git repository in the current folder.
git status	Shows modified, added, and untracked files.
git add <file>	Stages specific file changes for the next commit.
git add .	Stages ALL changes in the directory.
git commit -m "message"	Saves staged changes locally with a message.
git push origin <branch>	Uploads local commits to GitHub.
git pull origin <branch>	Fetches and merges latest commits from GitHub.
git log --oneline	Lists your history in a compact single-line view.
git remote -v	Lists remote server links connected to this project.
git config --list	Displays all configurations currently active in the directory.
git diff <file>	Shows modifications made to a file that are not yet staged.
git checkout -b <branch>	Creates and switches to a brand new branch.
git merge <branch>	Merges changes from the specified branch into your active branch.
git tag <tagname>	Bookmarks a commit with an immutable checkpoint tag.

Congratulations on your Git Mastery!

By understanding the principles behind configuration, staging, remote management, and tagging, you are adopting professional DevOps standards that ensure clean repo histories, secure keys, and highly functional code deployments.